

HEALTH INSURANCE POLICY & CLAIMS MANAGEMENT SYSTEM



Name: Tanishqa B

Roll No: 25WU0102285

Course: Database Management System (DBMS)

Faculty Name: Dr. Kiranmayee

Academic Year: 2026-27

Table of Contents

1. Cover Page
2. Abstract
3. Introduction and Problem Statement
4. Objectives and Scope
5. Software and Hardware Requirements
6. ER Diagram
7. Relational Schema and Normalization
8. Data Dictionary
9. SQL Commands Used (DDL, DML) with Sample Outputs
10. Queries with Outputs, including Presentation-II
11. UI Design and Screenshots
12. Implementation Details
13. Testing: Test Cases and Results
14. Conclusion and Future Enhancements
15. References
16. Appendix: GitHub Repository Link

2. Abstract

Health insurance administration involves customers, dependants, insurance plans, policies, premiums, hospitals, claims, treatment records, assessments and settlements. Maintaining these records in separate spreadsheets can introduce duplicate entries, inconsistent identifiers and delays in processing claims. A relational database provides a shared structure for storing these related records and retrieving them consistently.

The Health Insurance Policy and Claims Management System (HIPCMS) is a DBMS course project that connects policy administration with the claims workflow. The final MySQL implementation contains fifteen tables with primary keys, foreign keys, UNIQUE, NOT NULL, DEFAULT and CHECK constraints. Nine triggers implement selected business rules and status changes. The sample dataset includes fifteen customers, fifteen policies and fifteen claims, together with supporting records.

A Python Tkinter desktop interface uses MySQL Connector/Python to display, insert and delete records. Parameterized statements pass form values to MySQL, and the interface displays database errors and an SQL activity log. Queries demonstrate joins, filtering, grouping, aggregation and outstanding-payment calculations. Supplied screenshots document customer insertion and deletion, integrity checks and a hospital-wise claims query.

Keywords: DBMS, MySQL, Python, Tkinter, health insurance, claims management, relational schema, normalization, SQL.

3. Introduction and Problem Statement

3.1 Introduction

A health insurance policy defines financial coverage for an individual or family over a specified period. Customers pay premiums, while claims record requests for reimbursement or settlement of treatment expenses. Each claim must be associated with a policy and hospital, and may refer to a dependant. Treatment lines, supporting documents, assessments, decisions, settlements and appeals provide further detail about the claim.

HIPCMS models these relationships using separate tables linked by identifiers. A central database makes it possible to follow a policy from issuance to premium collection, and a claim from submission to decision and payment. The project focuses on database design and a local administration interface.

3.2 Problem Statement

- Disconnected records repeat customer and policy details and make updates inconsistent.
- Duplicate claims and invalid references can be entered when validation is manual.
- Premium dues and payment receipts are difficult to reconcile without linked records.
- Claims require a traceable relationship between treatment, assessment, decision and settlement.
- Management needs reports on active-status policies, pending decisions, hospital claims and unpaid premiums.

The problem is to design and implement an integrated relational database that stores these records, enforces defined integrity rules, and supports useful queries through an understandable interface.

4. Objectives and Scope

4.1 Objectives

- Identify the entities and relationships required for policy and claims administration.
- Map the conceptual ER design to a relational schema and explain normalization up to 3NF.
- Implement the schema and sample records in MySQL with keys, constraints and triggers.
- Provide a desktop interface to view, insert and delete records across the fifteen tables.
- Demonstrate SQL queries with outputs, including the query assigned during Presentation-II.
- Document observed behaviour and identify improvements to validation and workflow handling.

4.2 Scope

The database covers users, customers, dependants, plans, policies, premium schedules, payments, hospitals, claims, treatments, document references, assessments, decisions, settlements and appeals. The administration UI supports table browsing, form-based insertion, confirmed deletion, database refresh and reports.

Review I proposed several operational roles and policy renewal features. The delivered desktop application currently admits ADMIN users and uses grouped table-management tabs; separate customer, finance and assessor portals and dedicated renewal workflows are future extensions.

Out of scope: real insurer integrations, automatic medical eligibility decisions, payment gateways, document-file storage, online customer access and production deployment. The stored document reference is a path or reference string, not an uploaded PDF.

5. Software and Hardware Requirements

5.1 Software

Component	Specification / use
Operating system	macOS (project setup); Tkinter application
Language	Python 3.13; screenshot shows Python 3.13.16
Database	MySQL Server, compatible with supplied MySQL 8.0+ SQL
SQL client	MySQL Workbench
GUI	Tkinter and ttk, included in the official Python macOS installer
Connector	mysql-connector-python; setup screenshot shows 9.7.0
Database name / port	hipcms / 3306
Repository	GitHub
Design material	Review-I and Review-II presentations; supplied ER diagram

5.2 Practical hardware requirements

Component	Recommended for this coursework
Processor	Modern Intel or Apple Silicon processor
Memory	8 GB RAM recommended
Storage	At least 1 GB free for project tools and sample data
Display	1280 × 800 or larger recommended
Input / network	Keyboard and pointing device; internet for installation and GitHub

Hardware values are practical recommendations, not measured minimum requirements. The server version and exact Mac hardware were not recorded in the supplied evidence.

6. ER Diagram

Figure 6.1 reproduces the conceptual ER diagram from Review II. It describes the initial design; the final SQL refinements are explained in Section 7.2. The authoritative implemented relationships are listed on the following page.

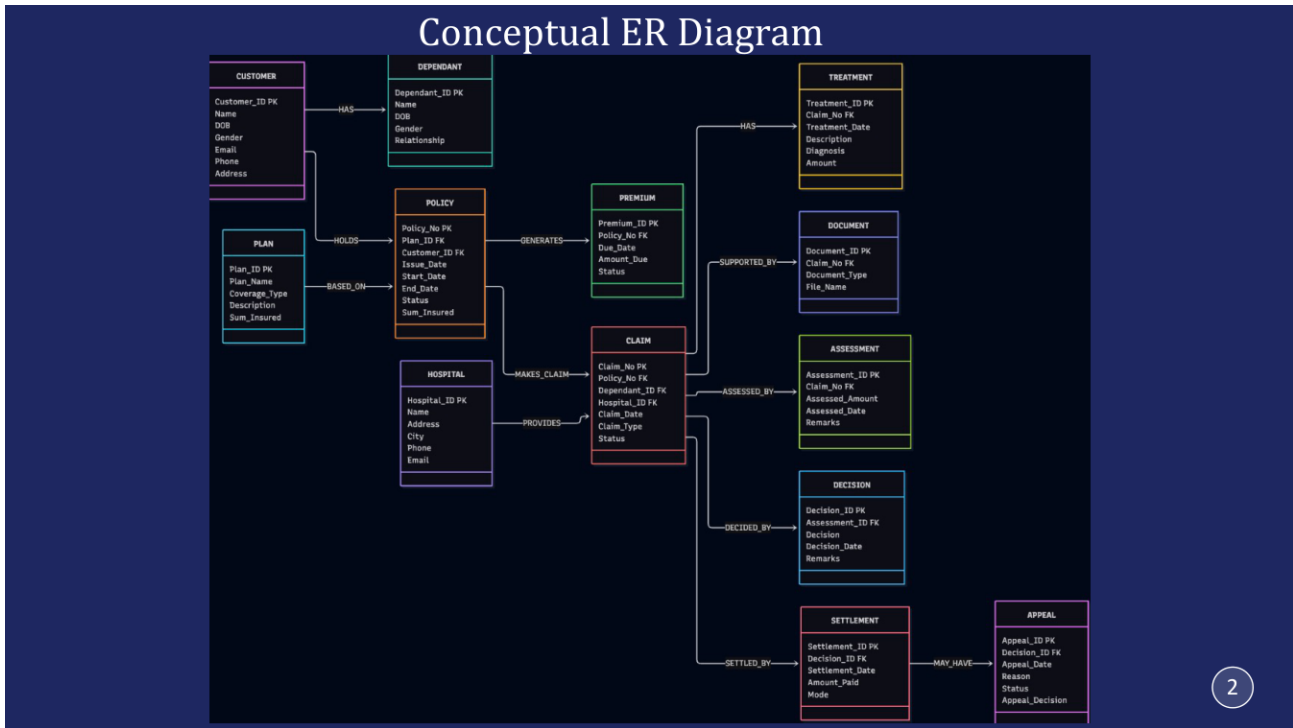


Figure 6.1: Original conceptual ER diagram supplied in Review II.

6.1 Implemented Relationships and Cardinality

Parent	Referencing column	Cardinality	ON DELETE
customer	dependant.customer_id	1 : many	CASCADE
customer	policy.customer_id	1 : many	RESTRICT
plan_master	policy.plan_id	1 : many	RESTRICT
users	policy.issued_by	1 : many	SET NULL
policy	premium.policy_id	1 : many	CASCADE
premium	payment.premium_id	1 : many	CASCADE
policy	claim.policy_id	1 : many	RESTRICT
dependant	claim.dependant_id	1 : many	SET NULL
hospital	claim.hospital_id	1 : many	RESTRICT
users	claim.submitted_by	1 : many	SET NULL
claim	treatment.claim_id	1 : many	CASCADE
claim	claim_document.claim_id	1 : many	CASCADE
claim	assessment.claim_id	1 : many	CASCADE
users	assessment.assessor_id	1 : many	RESTRICT
claim	decision.claim_id	1 : many	CASCADE
users	decision.decided_by	1 : many	RESTRICT
claim	settlement.claim_id	1 : 0..1	CASCADE
claim	appeal.claim_id	1 : many	CASCADE

Cardinality is stated from parent to children. A child has one parent for a NOT NULL foreign key, or zero/one for a nullable foreign key. The UNIQUE constraint on settlement.claim_id permits at most one settlement per claim. Assessments, decisions and appeals are not limited to one row per claim by the final DDL.

7. Relational Schema and Normalization

7.1 Final relations

USERS (user_id [PK], username, password_hash, full_name, role, is_active, created_at)

CUSTOMER (customer_id [PK], first_name, last_name, dob, gender, id_proof_number, phone, email, address_line, city, state, pincode, created_at)

DEPENDANT (dependant_id [PK], customer_id [FK], first_name, last_name, dob, gender, relationship)

PLAN_MASTER (plan_id [PK], plan_name, plan_type, coverage_amount, premium_amount, validity_months, description, is_active)

POLICY (policy_id [PK], policy_number, customer_id [FK], plan_id [FK], sum_insured, start_date, end_date, status, issued_by [FK], created_at)

PREMIUM (premium_id [PK], policy_id [FK], due_date, amount_due, status)

PAYMENT (payment_id [PK], premium_id [FK], payment_date, amount_paid, payment_mode, transaction_ref)

HOSPITAL (hospital_id [PK], hospital_name, address_line, city, state, network_type, contact_number)

CLAIM (claim_id [PK], claim_number, policy_id [FK], dependant_id [FK], hospital_id [FK], claim_date, claimed_amount, status, submitted_by [FK], created_at)

TREATMENT (treatment_id [PK], claim_id [FK], treatment_name, diagnosis, admission_date, discharge_date, treatment_cost)

CLAIM_DOCUMENT (document_id [PK], claim_id [FK], document_type, file_reference, upload_date)

ASSESSMENT (assessment_id [PK], claim_id [FK], assessor_id [FK], assessment_date, assessed_amount, remarks)

DECISION (decision_id [PK], claim_id [FK], decision_date, decision_status, approved_amount, decided_by [FK])

SETTLEMENT (settlement_id [PK], claim_id [FK], settlement_date, settled_amount, settlement_mode, transaction_ref)

APPEAL (appeal_id [PK], claim_id [FK], appeal_date, reason, appeal_status, resolution_date, resolution_remarks)

7.2 Refinements and Normalization

Changes from Review II

- USERS was added for application authentication and references to issuers, assessors and decision makers. PLAN and DOCUMENT became plan_master and claim_document.
- The final policy and claim tables use generated integer primary keys; their human-readable numbers remain UNIQUE alternate keys.
- The initial schema linked DECISION to ASSESSMENT. Final SQL links decision directly to claim and users. Settlement and appeal also reference claim directly.
- Customer names and addresses were expanded into separate fields. Treatment records use admission/discharge dates and treatment_cost.
- The final implementation includes premium payments, audit timestamps, indexes, CHECK constraints and nine triggers.

First Normal Form (1NF)

Each column stores an atomic value for the project's use, each row has a primary key, and repeating details are represented as separate rows. For example, multiple dependants of one customer are stored in dependant instead of a list inside customer.

Second Normal Form (2NF)

Attributes describe the complete key of their relation. The primary keys are single identifiers. The composite alternate key (policy_id, due_date) identifies a premium instalment; its amount and status describe that complete instalment rather than only a policy or a date.

Third Normal Form (3NF)

Under the stated domain assumptions, non-key attributes describe the entity or event identified by a candidate key. Customer contact details are stored in customer, plan definitions in plan_master, and hospital details in hospital. Policies and claims store foreign keys rather than repeating those descriptive details, reducing insertion, update and deletion anomalies.

Policy sum_insured is treated as the contracted coverage snapshot and is not assumed to be determined solely by plan_id. Pincode is stored as part of the address without an enforced pincode-to-city/state dependency. If that dependency is required, a separate postal-area relation would be needed. A blanket BCNF claim is not made. Stored workflow statuses also need consistency rules; normalization alone does not keep them synchronized.

8. Data Dictionary – USERS

Application identities and roles. PK = primary key; FK = foreign key. All identifier primary keys use AUTO_INCREMENT.

Column	Type	Column constraints	Meaning
user_id	INT	AUTO_INCREMENT PRIMARY KEY	User id
username	VARCHAR(50)	NOT NULL UNIQUE	Username
password_hash	VARCHAR(255)	NOT NULL	Password hash
full_name	VARCHAR(100)	NOT NULL	Full name
role	VARCHAR(20)	NOT NULL DEFAULT 'AGENT'	Role
is_active	TINYINT(1)	NOT NULL DEFAULT 1	Is active
created_at	TIMESTAMP	DEFAULT CURRENT_TIMESTAMP	Created at

Table-level constraints

```
CONSTRAINT chk_user_role CHECK (role IN ('ADMIN','AGENT','ASSESSOR','CUSTOMER'))
```

8.2 CUSTOMER

Policyholder identity and contact details. PK = primary key; FK = foreign key. All identifier primary keys use AUTO_INCREMENT.

Column	Type	Column constraints	Meaning
customer_id	INT	AUTO_INCREMENT PRIMARY KEY	Customer id
first_name	VARCHAR(50)	NOT NULL	First name
last_name	VARCHAR(50)	NOT NULL	Last name
dob	DATE	NOT NULL	Dob
gender	CHAR(1)	NOT NULL	Gender
id_proof_number	VARCHAR(20)	NOT NULL UNIQUE	Id proof number
phone	VARCHAR(15)	NOT NULL	Phone
email	VARCHAR(100)	NOT NULL UNIQUE	Email
address_line	VARCHAR(150)	NOT NULL	Address line
city	VARCHAR(50)	NOT NULL	City
state	VARCHAR(50)	NOT NULL	State
pincode	VARCHAR(10)	NOT NULL	Pincode
created_at	TIMESTAMP	DEFAULT CURRENT_TIMESTAMP	Created at

Table-level constraints

```
CONSTRAINT chk_customer_gender CHECK (gender IN ('M','F','O'))
```

8.3 DEPENDANT

Family members linked to a customer. PK = primary key; FK = foreign key. All identifier primary keys use AUTO_INCREMENT.

Column	Type	Column constraints	Meaning
dependant_id	INT	AUTO_INCREMENT PRIMARY KEY	Dependant id
customer_id	INT	NOT NULL; FK → customer.customer_id	Customer id
first_name	VARCHAR(50)	NOT NULL	First name
last_name	VARCHAR(50)	NOT NULL	Last name
dob	DATE	NOT NULL	Dob
gender	CHAR(1)	NOT NULL	Gender
relationship	VARCHAR(20)	NOT NULL	Relationship

Table-level constraints

```
CONSTRAINT fk_dependant_customer FOREIGN KEY (customer_id)
    REFERENCES customer(customer_id) ON DELETE CASCADE,
CONSTRAINT chk_dependant_gender CHECK (gender IN ('M', 'F', 'O')),
CONSTRAINT chk_dependant_relationship CHECK (
    relationship IN ('SPOUSE', 'SON', 'DAUGHTER', 'FATHER', 'MOTHER', 'OTHER')
)
```

8.4 PLAN_MASTER

Insurance products and standard pricing. PK = primary key; FK = foreign key. All identifier primary keys use AUTO_INCREMENT.

Column	Type	Column constraints	Meaning
plan_id	INT	AUTO_INCREMENT PRIMARY KEY	Plan id
plan_name	VARCHAR(100)	NOT NULL UNIQUE	Plan name
plan_type	VARCHAR(20)	NOT NULL	Plan type
coverage_amount	DECIMAL(12,2)	NOT NULL	Coverage amount
premium_amount	DECIMAL(10,2)	NOT NULL	Premium amount
validity_months	INT	NOT NULL DEFAULT 12	Validity months
description	VARCHAR(255)	NULL permitted	Description
is_active	TINYINT(1)	NOT NULL DEFAULT 1	Is active

Table-level constraints

```
CONSTRAINT chk_plan_type CHECK (plan_type IN  
( 'INDIVIDUAL', 'FAMILY_FLOATER', 'SENIOR_CITIZEN', 'GROUP' )),  
    CONSTRAINT chk_plan_coverage CHECK (coverage_amount > 0),  
    CONSTRAINT chk_plan_premium CHECK (premium_amount > 0)
```

8.5 POLICY

Issued insurance coverage contracts. PK = primary key; FK = foreign key. All identifier primary keys use AUTO_INCREMENT.

Column	Type	Column constraints	Meaning
policy_id	INT	AUTO_INCREMENT PRIMARY KEY	Policy id
policy_number	VARCHAR(20)	NOT NULL UNIQUE	Policy number
customer_id	INT	NOT NULL; FK → customer.customer_id	Customer id
plan_id	INT	NOT NULL; FK → plan_master.plan_id	Plan id
sum_insured	DECIMAL(12,2)	NOT NULL	Sum insured
start_date	DATE	NOT NULL	Start date
end_date	DATE	NOT NULL	End date
status	VARCHAR(15)	NOT NULL DEFAULT 'ACTIVE'	Status
issued_by	INT	NULL permitted; FK → users.user_id	Issued by
created_at	TIMESTAMP	DEFAULT CURRENT_TIMESTAMP	Created at

Table-level constraints

```
CONSTRAINT fk_policy_customer FOREIGN KEY (customer_id)
    REFERENCES customer(customer_id) ON DELETE RESTRICT,
CONSTRAINT fk_policy_plan FOREIGN KEY (plan_id)
    REFERENCES plan_master(plan_id) ON DELETE RESTRICT,
CONSTRAINT fk_policy_issuer FOREIGN KEY (issued_by)
    REFERENCES users(user_id) ON DELETE SET NULL,
CONSTRAINT chk_policy_status CHECK (status IN ('ACTIVE', 'LAPSED', 'CANCELLED', 'EXPIRED')),
CONSTRAINT chk_policy_dates CHECK (end_date > start_date),
CONSTRAINT chk_policy_sum_insured CHECK (sum_insured > 0)
```

8.6 PREMIUM

Amounts due for policy instalments. PK = primary key; FK = foreign key. All identifier primary keys use AUTO_INCREMENT.

Column	Type	Column constraints	Meaning
premium_id	INT	AUTO_INCREMENT PRIMARY KEY	Premium id
policy_id	INT	NOT NULL; FK → policy.policy_id	Policy id
due_date	DATE	NOT NULL	Due date
amount_due	DECIMAL(10,2)	NOT NULL	Amount due
status	VARCHAR(15)	NOT NULL DEFAULT 'PENDING'	Status

Table-level constraints

```
CONSTRAINT fk_premium_policy FOREIGN KEY (policy_id)
    REFERENCES policy(policy_id) ON DELETE CASCADE,
CONSTRAINT chk_premium_status CHECK (status IN ('PENDING', 'PAID', 'OVERDUE')),
CONSTRAINT chk_premium_amount CHECK (amount_due > 0),
CONSTRAINT uq_premium_policy_due UNIQUE (policy_id, due_date)
```

8.7 PAYMENT

Receipts against premium instalments. PK = primary key; FK = foreign key. All identifier primary keys use AUTO_INCREMENT.

Column	Type	Column constraints	Meaning
payment_id	INT	AUTO_INCREMENT PRIMARY KEY	Payment id
premium_id	INT	NOT NULL; FK → premium.premium_id	Premium id
payment_date	DATE	NOT NULL	Payment date
amount_paid	DECIMAL(10,2)	NOT NULL	Amount paid
payment_mode	VARCHAR(15)	NOT NULL	Payment mode
transaction_ref	VARCHAR(30)	NOT NULL UNIQUE	Transaction ref

Table-level constraints

```
CONSTRAINT fk_payment_premium FOREIGN KEY (premium_id)
    REFERENCES premium(premium_id) ON DELETE CASCADE,
CONSTRAINT chk_payment_mode CHECK (payment_mode IN
('CASH', 'CARD', 'UPI', 'NETBANKING', 'CHEQUE')),
CONSTRAINT chk_payment_amount CHECK (amount_paid > 0)
```

8.8 HOSPITAL

Hospital contact and network information. PK = primary key; FK = foreign key. All identifier primary keys use AUTO_INCREMENT.

Column	Type	Column constraints	Meaning
hospital_id	INT	AUTO_INCREMENT PRIMARY KEY	Hospital id
hospital_name	VARCHAR(100)	NOT NULL	Hospital name
address_line	VARCHAR(150)	NOT NULL	Address line
city	VARCHAR(50)	NOT NULL	City
state	VARCHAR(50)	NOT NULL	State
network_type	VARCHAR(15)	NOT NULL	Network type
contact_number	VARCHAR(15)	NOT NULL	Contact number

Table-level constraints

```
CONSTRAINT chk_hospital_network CHECK (network_type IN ('NETWORK','NON_NETWORK'))
```

8.9 CLAIM

Requests against policy coverage. PK = primary key; FK = foreign key. All identifier primary keys use AUTO_INCREMENT.

Column	Type	Column constraints	Meaning
claim_id	INT	AUTO_INCREMENT PRIMARY KEY	Claim id
claim_number	VARCHAR(20)	NOT NULL UNIQUE	Claim number
policy_id	INT	NOT NULL; FK → policy.policy_id	Policy id
dependant_id	INT	NULL; FK → dependant.dependant_id	Dependant id
hospital_id	INT	NOT NULL; FK → hospital.hospital_id	Hospital id
claim_date	DATE	NOT NULL	Claim date
claimed_amount	DECIMAL(12,2)	NOT NULL	Claimed amount
status	VARCHAR(20)	NOT NULL DEFAULT 'SUBMITTED'	Status
submitted_by	INT	NULL permitted; FK → users.user_id	Submitted by
created_at	TIMESTAMP	DEFAULT CURRENT_TIMESTAMP	Created at

Table-level constraints

```
CONSTRAINT fk_claim_policy FOREIGN KEY (policy_id)
    REFERENCES policy(policy_id) ON DELETE RESTRICT,
CONSTRAINT fk_claim_dependant FOREIGN KEY (dependant_id)
    REFERENCES dependant(dependant_id) ON DELETE SET NULL,
CONSTRAINT fk_claim_hospital FOREIGN KEY (hospital_id)
    REFERENCES hospital(hospital_id) ON DELETE RESTRICT,
CONSTRAINT fk_claim_user FOREIGN KEY (submitted_by)
    REFERENCES users(user_id) ON DELETE SET NULL,
CONSTRAINT chk_claim_status CHECK (
    status IN
('SUBMITTED', 'VERIFIED', 'UNDER_ASSESSMENT', 'APPROVED', 'REJECTED', 'SETTLED', 'APPEALED')
),
CONSTRAINT chk_claim_amount CHECK (claimed_amount > 0)
```

8.10 TREATMENT

Treatment and admission details for claims. PK = primary key; FK = foreign key. All identifier primary keys use AUTO_INCREMENT.

Column	Type	Column constraints	Meaning
treatment_id	INT	AUTO_INCREMENT PRIMARY KEY	Treatment id
claim_id	INT	NOT NULL; FK → claim.claim_id	Claim id
treatment_name	VARCHAR(100)	NOT NULL	Treatment name
diagnosis	VARCHAR(150)	NOT NULL	Diagnosis
admission_date	DATE	NOT NULL	Admission date
discharge_date	DATE	NOT NULL	Discharge date
treatment_cost	DECIMAL(12,2)	NOT NULL	Treatment cost

Table-level constraints

```
CONSTRAINT fk_treatment_claim FOREIGN KEY (claim_id)
    REFERENCES claim(claim_id) ON DELETE CASCADE,
CONSTRAINT chk_treatment_dates CHECK (discharge_date >= admission_date),
CONSTRAINT chk_treatment_cost CHECK (treatment_cost > 0)
```

8.11 CLAIM_DOCUMENT

Supporting document reference metadata. PK = primary key; FK = foreign key. All identifier primary keys use AUTO_INCREMENT.

Column	Type	Column constraints	Meaning
document_id	INT	AUTO_INCREMENT PRIMARY KEY	Document id
claim_id	INT	NOT NULL; FK → claim.claim_id	Claim id
document_type	VARCHAR(30)	NOT NULL	Document type
file_reference	VARCHAR(255)	NOT NULL	File reference
upload_date	DATE	NOT NULL	Upload date

Table-level constraints

```
CONSTRAINT fk_document_claim FOREIGN KEY (claim_id)
    REFERENCES claim(claim_id) ON DELETE CASCADE,
CONSTRAINT chk_document_type CHECK (
    document_type IN
    ('PRESCRIPTION', 'DISCHARGE_SUMMARY', 'BILL', 'LAB_REPORT', 'ID_PROOF', 'OTHER')
)
```

8.12 ASSESSMENT

Assessment amounts and assessor remarks. PK = primary key; FK = foreign key. All identifier primary keys use AUTO_INCREMENT.

Column	Type	Column constraints	Meaning
assessment_id	INT	AUTO_INCREMENT PRIMARY KEY	Assessment id
claim_id	INT	NOT NULL; FK → claim.claim_id	Claim id
assessor_id	INT	NOT NULL; FK → users.user_id	Assessor id
assessment_date	DATE	NOT NULL	Assessment date
assessed_amount	DECIMAL(12,2)	NOT NULL	Assessed amount
remarks	VARCHAR(255)	NULL permitted	Remarks

Table-level constraints

```
CONSTRAINT fk_assessment_claim FOREIGN KEY (claim_id)
    REFERENCES claim(claim_id) ON DELETE CASCADE,
CONSTRAINT fk_assessment_assessor FOREIGN KEY (assessor_id)
    REFERENCES users(user_id) ON DELETE RESTRICT,
CONSTRAINT chk_assessment_amount CHECK (assessed_amount >= 0)
```

8.13 DECISION

Claim approval or rejection decisions. PK = primary key; FK = foreign key. All identifier primary keys use AUTO_INCREMENT.

Column	Type	Column constraints	Meaning
decision_id	INT	AUTO_INCREMENT PRIMARY KEY	Decision id
claim_id	INT	NOT NULL; FK → claim.claim_id	Claim id
decision_date	DATE	NOT NULL	Decision date
decision_status	VARCHAR(20)	NOT NULL	Decision status
approved_amount	DECIMAL(12,2)	NOT NULL DEFAULT 0	Approved amount
decided_by	INT	NOT NULL; FK → users.user_id	Decided by

Table-level constraints

```
CONSTRAINT fk_decision_claim FOREIGN KEY (claim_id)
    REFERENCES claim(claim_id) ON DELETE CASCADE,
CONSTRAINT fk_decision_user FOREIGN KEY (decided_by)
    REFERENCES users(user_id) ON DELETE RESTRICT,
CONSTRAINT chk_decision_status CHECK (
    decision_status IN ('APPROVED', 'REJECTED', 'PARTIALLY_APPROVED')
),
CONSTRAINT chk_decision_amount CHECK (approved_amount >= 0)
```

8.14 SETTLEMENT

Final payout record per claim. PK = primary key; FK = foreign key. All identifier primary keys use AUTO_INCREMENT.

Column	Type	Column constraints	Meaning
settlement_id	INT	AUTO_INCREMENT PRIMARY KEY	Settlement id
claim_id	INT	NOT NULL UNIQUE; FK → claim.claim_id	Claim id
settlement_date	DATE	NOT NULL	Settlement date
settled_amount	DECIMAL(12,2)	NOT NULL	Settled amount
settlement_mode	VARCHAR(15)	NOT NULL	Settlement mode
transaction_ref	VARCHAR(30)	NOT NULL UNIQUE	Transaction ref

Table-level constraints

```
CONSTRAINT fk_settlement_claim FOREIGN KEY (claim_id)
    REFERENCES claim(claim_id) ON DELETE CASCADE,
CONSTRAINT chk_settlement_mode CHECK (settlement_mode IN ('NEFT','CHEQUE','UPI')),
CONSTRAINT chk_settlement_amount CHECK (settled_amount > 0)
```

8.15 APPEAL

Appeal submissions and resolution details. PK = primary key; FK = foreign key. All identifier primary keys use AUTO_INCREMENT.

Column	Type	Column constraints	Meaning
appeal_id	INT	AUTO_INCREMENT PRIMARY KEY	Appeal id
claim_id	INT	NOT NULL; FK → claim.claim_id	Claim id
appeal_date	DATE	NOT NULL	Appeal date
reason	VARCHAR(255)	NOT NULL	Reason
appeal_status	VARCHAR(15)	NOT NULL DEFAULT 'PENDING'	Appeal status
resolution_date	DATE	NULL permitted	Resolution date
resolution_remarks	VARCHAR(255)	NULL permitted	Resolution remarks

Table-level constraints

```
CONSTRAINT fk_appeal_claim FOREIGN KEY (claim_id)
    REFERENCES claim(claim_id) ON DELETE CASCADE,
CONSTRAINT chk_appeal_status CHECK (appeal_status IN ('PENDING', 'RESOLVED', 'REJECTED'))
```

9. SQL Commands Used – DDL

DDL defines the database, tables, keys, constraints, indexes and triggers. The following statements are excerpts from the supplied final SQL. Example success messages are illustrative; no new live MySQL execution is claimed in this report.

```
CREATE DATABASE IF NOT EXISTS hipcms CHARACTER SET utf8mb4;
USE hipcms;

CREATE TABLE dependant (

    dependant_id    INT AUTO_INCREMENT PRIMARY KEY,
    customer_id     INT NOT NULL,
    first_name      VARCHAR(50) NOT NULL,
    last_name       VARCHAR(50) NOT NULL,
    dob            DATE NOT NULL,
    gender          CHAR(1) NOT NULL,
    relationship     VARCHAR(20) NOT NULL,
    CONSTRAINT fk_dependant_customer FOREIGN KEY (customer_id)
        REFERENCES customer(customer_id) ON DELETE CASCADE,
    CONSTRAINT chk_dependant_gender CHECK (gender IN ('M','F','O')),
    CONSTRAINT chk_dependant_relationship CHECK (
        relationship IN ('SPOUSE','SON','DAUGHTER','FATHER','MOTHER','OTHER')
    )
);
```

Expected result: the schema is selected and dependant is created when the parent customer table exists. A successful CREATE TABLE normally reports “Query OK”.

```
CREATE INDEX idx_customer_name ON customer (last_name, first_name);
SHOW TABLES;
```

Expected tables	(15 in total)	
users	customer	dependant
plan_master	policy	premium
payment	hospital	claim
treatment	claim_document	assessment
decision	settlement	appeal

9.1 Business-Rule Triggers

Trigger	Implemented behaviour
trg_customer_dob_past	Before INSERT: reject a birth date today or in the future.
trg_claim_active_coverage	Before INSERT: require ACTIVE policy status and a claim date inside coverage dates.
trg_claim_no_duplicate	Before INSERT: block same policy/hospital/date if a non-rejected claim exists.
trg_treatment_within_validity	Before INSERT: enforce treatment dates within the related policy period.
trg_settlement_within_limits	Before INSERT: enforce sum insured and, when positive, the maximum approved decision amount.
trg_after_decision_update_claim	After INSERT: map decision to APPROVED or REJECTED claim status.
trg_after_settlement_update_claim	After INSERT: mark claim SETTLED.
trg_after_payment_update_premium	After INSERT: mark premium PAID.
trg_after_appeal_update_claim	After INSERT: mark claim APPEALED.

Implementation limits

These triggers run on INSERT only. Deleting a payment, decision or appeal does not reverse its prior status update. A payment marks an instalment PAID without checking cumulative payment amounts. Settlement validation uses MAX(approved_amount), and its approved-amount check is skipped when that maximum is zero; it does not require an approved decision. These are documented limitations, not completed validation features.

9.2 SQL Commands Used – DML

The supplied sample script inserts records in parent-before-child order. The table below gives row counts derived from that script, before the Ananya demonstration insertion.

Table	Sample rows
users	5
customer	15
dependant	12
plan_master	5
policy	15
premium	20
payment	12
hospital	8
claim	15
treatment	15
claim_document	18
assessment	12
decision	12
settlement	7
appeal	3

Sample INSERT

```
INSERT INTO customer
(first_name, last_name, dob, gender, id_proof_number, phone,
email, address_line, city, state, pincode)
VALUES ('Ananya', 'Rao', '1996-04-12', 'F', 'AADH-9988-7766',
'9876543210', 'ananya.rao@example.com', '12 MG Road',
'Hyderabad', 'Telangana', '500001');
```

Observed screenshot output: one row inserted, customer_id = 16, customer count 15
→ 16, commit completed. Generated IDs can differ on later runs.

9.3 DML Verification and Deletion

```
SELECT customer_id, first_name, last_name, city
FROM customer WHERE email = 'ananya.rao@example.com';
```

customer_id	first_name	last_name	city
16	Ananya	Rao	Hyderabad

The example row above is visible in the supplied after-insertion UI screenshot.

UPDATE example

```
START TRANSACTION;
UPDATE customer SET phone = '9876543211'
WHERE email = 'ananya.rao@example.com';
SELECT first_name, phone FROM customer
WHERE email = 'ananya.rao@example.com';
ROLLBACK;
```

Illustrative expected output: if Ananya exists, her selected phone becomes 9876543211 inside the transaction; ROLLBACK restores the prior value. This UPDATE was not documented as a live test in the supplied screenshots.

DELETE demonstration

```
DELETE FROM customer WHERE customer_id = 16;
SELECT COUNT(*) AS customers FROM customer;
SELECT * FROM customer WHERE customer_id = 16;
```

Observed screenshot output: one row deleted; customer count returns to 15; the verification SELECT returns zero rows for customer 16. The deletion targets the demonstration row shown in that screenshot.

10. Queries with Outputs

Outputs below are calculated from the supplied sample INSERT statements; they are reproducible expected results rather than new MySQL execution screenshots. The Presentation-II output on the next dedicated page is supported by the supplied Workbench screenshot. Monetary values are in INR.

Query 1: Customer count

```
SELECT COUNT(*) AS total_customers FROM customer;
```

total_customers
15

Query 2: Policies with stored ACTIVE status

```
SELECT policy_number, customer_id, sum_insured  
FROM policy WHERE status = 'ACTIVE' ORDER BY policy_id;
```

policy_number	customer_id	sum_insured
POL-2025-0001	1	1000000
POL-2025-0002	2	300000
POL-2025-0004	4	300000
POL-2025-0005	5	1000000
POL-2025-0006	6	500000
POL-2025-0007	7	1000000
POL-2025-0008	8	300000
POL-2025-0009	9	2000000
POL-2025-0010	10	150000
POL-2025-0012	12	300000
POL-2025-0013	13	500000
POL-2025-0014	14	150000

ACTIVE here means the stored status. The historical dates do not establish that coverage is active today.

10. Queries with Outputs — continued

Query 3: Customer and policy join

```
SELECT c.first_name, c.last_name, p.policy_number
FROM customer c JOIN policy p ON c.customer_id=p.customer_id
ORDER BY p.policy_id LIMIT 5;
```

first_name	last_name	policy_number
Arun	Sharma	POL-2025-0001
Divya	Iyer	POL-2025-0002
Karthik	Reddy	POL-2025-0003
Sneha	Patil	POL-2025-0004
Mohammed	Ali	POL-2025-0005

Query 4: Pending decisions

```
SELECT c.claim_number, c.claimed_amount
FROM claim c WHERE NOT EXISTS
(SELECT 1 FROM decision d WHERE d.claim_id=c.claim_id)
ORDER BY c.claim_id;
```

claim_number	claimed_amount
CLM-2025-0003	120000
CLM-2025-0009	15000
CLM-2025-0015	60000

10. Queries with Outputs — continued

Query 5: Claims by hospital

```
SELECT h.hospital_name, COUNT(c.claim_id) AS claims
FROM hospital h JOIN claim c ON h.hospital_id=c.hospital_id
GROUP BY h.hospital_id,h.hospital_name ORDER BY h.hospital_id;
```

hospital_name	claims
Apollo Speciality Hospital	4
Care Hospitals	3
Ruby Hall Clinic	1
City General Hospital	1
Manipal Hospital	1
Lilavati Hospital	2
Fortis Escorts Heart Institute	1
KIMS Hospital	2

Query 6: Payments by mode

```
SELECT payment_mode,COUNT(*) AS payments,SUM(amount_paid) AS total
FROM payment GROUP BY payment_mode ORDER BY payment_mode;
```

payment_mode	payments	total
CARD	2	44000
CASH	1	22000
CHEQUE	2	36000
NETBANKING	2	23500
UPI	5	79000

10. Queries with Outputs — continued

Query 7: Outstanding premium instalments

```
SELECT p.premium_id,p.amount_due,  
       p.amount_due-COALESCE(SUM(x.amount_paid),0) AS balance  
FROM premium p LEFT JOIN payment x ON p.premium_id=x.premium_id  
GROUP BY p.premium_id,p.amount_due  
HAVING balance>0 ORDER BY p.premium_id;
```

premium_id	amount_due	balance
8	8500	8500
12	8500	8500
14	4500	4500
16	22000	22000
17	8500	8500
18	22000	22000
19	15000	15000
20	18000	18000

Query 8: Approved decisions without settlement

```
SELECT c.claim_number,d.approved_amount  
FROM claim c JOIN decision d ON d.claim_id=c.claim_id  
WHERE d.decision_status IN ('APPROVED','PARTIALLY_APPROVED')  
AND NOT EXISTS (SELECT 1 FROM settlement s  
                 WHERE s.claim_id=c.claim_id) ORDER BY c.claim_id;
```

claim_number	approved_amount
CLM-2025-0001	80000
CLM-2025-0004	55000
CLM-2025-0008	220000
CLM-2025-0011	50000

The sample has one decision per decided claim. With decision history, filter to the latest decision before calculating this report.

10. Queries with Outputs — continued

Query 9: Total settlements

```
SELECT COUNT(*) AS settlements, SUM(settled_amount) AS total_paid
FROM settlement;
```

settlements	total_paid
7	282000

Query 10: Decision approval rate

```
SELECT COUNT(*) AS decisions,
ROUND(100.0*SUM(CASE WHEN decision_status IN
('APPROVED', 'PARTIALLY_APPROVED') THEN 1 ELSE 0 END)
/COUNT(*),2) AS approval_percent FROM decision;
```

decisions	approval_percent
12	91.67

The sample has one decision per decided claim. With decision history, filter to the latest decision before calculating this report.

10.1 Query Given During Presentation-II

Task: display the number of claims associated with hospitals whose names contain KIMS. The query combines an inner join, pattern matching, COUNT and GROUP BY.

```
SELECT h.hospital_name,  
       COUNT(cl.claim_id) AS number_of_claims  
FROM hospital h  
JOIN claim cl ON h.hospital_id = cl.hospital_id  
WHERE h.hospital_name LIKE '%KIMS%'  
GROUP BY h.hospital_id, h.hospital_name;
```

hospital_name	number_of_claims
KIMS Hospital	2

Hospital 8 (KIMS Hospital) is referenced by claims CLM-2025-0009 and CLM-2025-0012 in the supplied data. The observed Workbench result is two claims.

```
184  
185 • SELECT h.hospital_name,  
186         COUNT(cl.claim_id) AS number_of_claims  
187     FROM hospital h  
188     JOIN claim cl ON h.hospital_id = cl.hospital_id  
189     WHERE h.hospital_name LIKE '%KIMS%'  
190     GROUP BY h.hospital_id, h.hospital_name;
```

100% 41:190

Result Grid Filter Rows: Search Export:

hospital_name	number_of_clai...
KIMS Hospital	2

Figure 10.1: Supplied Workbench execution of the Presentation-II query.

11. UI Design and Screenshots

The supplied interface screenshots show a dark header, grouped tabs, record grids, insertion dialogs and an SQL activity log. They are reproduced as project evidence. The Mac Tkinter package recreates the main record-management screen; some secondary tab layouts differ from these reference screenshots.

HIPCMS System Administrator | ADMIN

Database Records Customers & Dependants Policies & Premiums Claims Appeals Reports

Table: customer View Records + Insert Record Delete Selected Verify in DB 15 row(s) in `customer`

customer_id (PK)	first_name	last_name	dob	gender	id_proof_number	phone	email	address_line	city
1	Arun	Sharma	1985-04-12	M	ID100001	9876500001	arun.sharma@example.com	12 MG Road	Chennai
2	Divya	Iyer	1990-08-25	F	ID100002	9876500002	divya.iyer@example.com	45 Anna Nagar	Chennai
3	Karthik	Reddy	1978-01-30	M	ID100003	9876500003	karthik.reddy@example.com	7 Jubilee Hills	Hyderabad
4	Sneha	Patil	1995-11-05	F	ID100004	9876500004	sneha.patil@example.com	23 FC Road	Pune
5	Mohammed	Ali	1982-06-18	M	ID100005	9876500005	mohammed.ali@example.com	9 Banjara Hills	Hyderabad
6	Lakshmi	Narayanan	1970-03-22	F	ID100006	9876500006	lakshmi.n@example.com	56 T Nagar	Chennai
7	Rajesh	Kumar	1988-09-14	M	ID100007	9876500007	rajesh.kumar@example.com	18 Indiranagar	Bengaluru
8	Priya	Venkatesh	1992-12-02	F	ID100008	9876500008	priya.venkatesh@example.com	27 Koramangala	Bengaluru
9	Anil	Kapoor	1975-07-19	M	ID100009	9876500009	anil.kapoor@example.com	5 Andheri West	Mumbai
10	Sunita	Rao	1983-02-27	F	ID100010	9876500010	sunita.rao@example.com	61 Bandra	Mumbai
11	Vikram	Singh	1980-10-09	M	ID100011	9876500011	vikram.singh@example.com	33 Connaught Place	New Delhi
12	Neha	Gupta	1993-05-16	F	ID100012	9876500012	neha.gupta@example.com	8 Karol Bagh	New Delhi
13	Suresh	Babu	1972-01-08	M	ID100013	9876500013	suresh.babu@example.com	14 RS Puram	Coimbatore
14	Kavya	Krishnan	1991-03-29	F	ID100014	9876500014	kavya.krishnan@example.com	21 MG Road	Kochi
15	Ramesh	Iyer	1968-11-11	M	ID100015	9876500015	ramesh.iyer@example.com	3 Anna Nagar	Madurai

SQL activity log - statements sent to the database

```
[18:23:20] Connected to MySQL root@127.0.0.1:3306 / hipcms
```

Ready. DB: MySQL root@127.0.0.1:3306 / hipcms

Figure 11.1: Customer records before insertion: 15 rows.

HIPCMS System Administrator | ADMIN

Database Records Customers & Dependants Policies & Premiums Claims Appeals Reports

Table: customer View Records + Insert Record Delete Selected Verify in DB 15 row(s) in `customer`

New row for table: customer

* = required. Leave optional fields blank to store NULL / the column default.
Filled automatically by MySQL: customer_id, created_at

first_name *	Ananya	varchar(50)
last_name *	Rao	varchar(50)
dob *	1996-04-12	date (YYYY-MM-DD)
gender *	F	CHECK: M/F/O
id_proof_number *	AADH-9988-7766	varchar(20)
phone *	9876543210	varchar(15)
email *	ananya.rao@example.com	varchar(100)
address_line *	12 MG Road	varchar(150)
city *	Hyderabad	varchar(50)
state *	Telangana	varchar(50)
zipcode *	500001	varchar(10)

INSERT Cancel

SQL activity log - statements sent to the database

```
[18:23:20] Connected to MySQL root@127.0.0.1:3306 / hipcms
```

Ready. DB: MySQL root@127.0.0.1:3306 / hipcms

Figure 11.2: Customer insertion form containing Ananya Rao's details.

11. UI Design and Screenshots – continued

HIPCMS System Administrator | ADMIN

Database Records Customers & Dependants Policies & Premiums Claims Appeals Reports

Table: customer View Records + Insert Record Delete Selected Verify in DB 16 row(s) in `customer`

customer_id (PK)	first_name	last_name	dob	gender	id_proof number	phone	email	address_line	city
1	Arun	Sharma	1985-04-12	M	ID100001	9876500001	arun.sharma@example.com	12 MG Road	Chennai
2	Divya	Iyer	1990-08-25	F	ID100002	9876500002	divya.iyer@example.com	45 Anna Nagar	Chennai
3	Karthik	Reddy	1978-01-30	M	ID100003	9876500003	karthik.reddy@example.com	7 Jubilee Hills	Hyderabad
4	Sneha	Patil	1995-11-05	F	ID100004	9876500004	sneha.patil@example.com	23 FC Road	Pune
5	Mohammed	Ali	1982-06-18	M	ID100005	9876500005	mohammed.ali@example.com	9 Banjara Hills	Hyderabad
6	Lakshmi	Narayanan	1970-03-22	F	ID100006	9876500006	lakshmi.n@example.com	56 T Nagar	Chennai
7	Rajesh	Kumar	1988-09-14	M	ID100007	9876500007	rajesh.kumar@example.com	18 Indiranagar	Bengaluru
8	Priya	Venkatesh	1992-12-02	F	ID100008	9876500008	priya.venkatesh@example.com	27 Koramangala	Bengaluru
9	Anil	Kapoor	1975-07-19	M	ID100009	9876500009	anil.kapoor@example.com	5 Andheri West	Mumbai
10	Sunita	Rao	1983-02-27	F	ID100010	9876500010	sunita.rao@example.com	61 Bandra	Mumbai
11	Vikram	Singh	1980-10-09	M	ID100011	9876500011	vikram.singh@example.com	33 Connaught Place	New Delhi
12	Neha	Gupta	1993-05-16	F	ID100012	9876500012	neha.gupta@example.com	8 Karol Bagh	New Delhi
13	Suresh	Babu	1972-01-08	M	ID100013	9876500013	suresh.babu@example.com	14 RS Puram	Coimbatore
14	Kavya	Krishnan	1991-03-29	F	ID100014	9876500014	kavya.krishnan@example.com	21 MG Road	Kochi
15	Ramesh	Iyer	1968-11-11	M	ID100015	9876500015	ramesh.iyer@example.com	3 Anna Nagar	Madurai
16	Ananya	Rao	1996-04-12	F	AADH-9988-7766	9876543210	ananya.rao@example.com	12 MG Road	Hyderabad

SQL activity log - statements sent to the database

```
[18:23:20] Connected to MySQL root@127.0.0.1:3306 / hipcms
[18:23:22] INSERT INTO customer (first_name, last_name, dob, gender, id_proof number, phone, email, address_line, city, state, pincode)
VALUES ('Ananya', 'Rao', '1996-04-12', 'F', 'AADH-9988-7766', '9876543210', 'ananya.rao@example.com', '12 MG Road', 'Hyderabad',
'Telangana', '500001');
[18:23:22] -> 1 row inserted, customer_id = 16 | COUNT(*) 15 -> 16 (COMMIT done)
```

Inserted into `customer`: customer_id=16. Row count 15 -> 16. DB: MySQL root@127.0.0.1:3306 / hipcms

Figure 11.3: After insertion: customer 16 appears and the count becomes 16.

HIPCMS System Administrator | ADMIN

Database Records Customers & Dependants Policies & Premiums Claims Appeals Reports

Table: customer View Records + Insert Record Delete Selected Verify in DB 15 row(s) in `customer`

customer_id (PK)	first_name	last_name	dob	gender	id_proof number	phone	email	address_line	city
1	Arun	Sharma	1985-04-12	M	ID100001	9876500001	arun.sharma@example.com	12 MG Road	Chennai
2	Divya	Iyer	1990-08-25	F	ID100002	9876500002	divya.iyer@example.com	45 Anna Nagar	Chennai
3	Karthik	Reddy	1978-01-30	M	ID100003	9876500003	karthik.reddy@example.com	7 Jubilee Hills	Hyderabad
4	Sneha	Patil	1995-11-05	F	ID100004	9876500004	sneha.patil@example.com	23 FC Road	Pune
5	Mohammed	Ali	1982-06-18	M	ID100005	9876500005	mohammed.ali@example.com	9 Banjara Hills	Hyderabad
6	Lakshmi	Narayanan	1970-03-22	F	ID100006	9876500006	lakshmi.n@example.com	56 T Nagar	Chennai
7	Rajesh	Kumar	1988-09-14	M	ID100007	9876500007	rajesh.kumar@example.com	18 Indiranagar	Bengaluru
8	Priya	Venkatesh	1992-12-02	F	ID100008	9876500008	priya.venkatesh@example.com	27 Koramangala	Bengaluru
9	Anil	Kapoor	1975-07-19	M	ID100009	9876500009	anil.kapoor@example.com	5 Andheri West	Mumbai
10	Sunita	Rao	1983-02-27	F	ID100010	9876500010	sunita.rao@example.com	61 Bandra	Mumbai
11	Vikram	Singh	1980-10-09	M	ID100011	9876500011	vikram.singh@example.com	33 Connaught Place	New Delhi
12	Neha	Gupta	1993-05-16	F	ID100012	9876500012	neha.gupta@example.com	8 Karol Bagh	New Delhi
13	Suresh	Babu	1972-01-08	M	ID100013	9876500013	suresh.babu@example.com	14 RS Puram	Coimbatore
14	Kavya	Krishnan	1991-03-29	F	ID100014	9876500014	kavya.krishnan@example.com	21 MG Road	Kochi
15	Ramesh	Iyer	1968-11-11	M	ID100015	9876500015	ramesh.iyer@example.com	3 Anna Nagar	Madurai

SQL activity log - statements sent to the database

```
[18:23:23] VALUES ('Dup', 'User', '1990-01-01', 'M', 'X-1', '9999999999', 'ananya.rao@example.com', 'x', 'x', 'x', '1');
[18:23:23] -> REJECTED by MySQL: Duplicate value - this must be unique (UNIQUE constraint). Duplicate entry 'ananya.rao@example.com' for
key 'customer_email'
[18:23:23] DELETE FROM customer WHERE customer_id = 16;
[18:23:23] -> 1 row(s) deleted | COUNT(*) 16 -> 15 (COMMIT done)
[18:23:24] [new connection] SELECT COUNT(*) FROM customer; -> 15
[18:23:24] [new connection] SELECT * FROM customer WHERE customer_id = 16;
[18:23:24] -> 0 rows: record does not exist in the database
```

Deleted customer_id=16 from `customer`. Row count 16 -> 15. DB: MySQL root@127.0.0.1:3306 / hipcms

Figure 11.4: After deletion: 15 rows remain; the log verifies zero rows for customer 16.

11. UI Design and Screenshots — continued

The screenshot shows the HIPCMS interface with the 'dependant' form open. The form is titled 'New row for table: dependant' and includes fields for customer_id, first_name, last_name, dob, gender, and relationship. The customer_id is set to '1 - Arun Sharma'. The first_name is 'Kiran', last_name is 'Rao', dob is '2015-06-01', gender is 'M', and relationship is 'SON'. There are 'INSERT' and 'Cancel' buttons at the bottom of the form. Below the form is an SQL activity log showing a sequence of queries and their results, including a 'DELETE FROM customer WHERE customer_id = 1;' query that was rejected by MySQL due to a foreign key constraint.

Table: dependant View Records + Insert Record Delete Selected Verify in DB 12 row(s) in 'dependant'

dependant_id (PK)	customer_id (FK)	first_name	last_name	dob	gender	relationship
1	1					
2	1					
3	2					
4	3					
5	5					
6	7					
7	7					
8	9					
9	11					
10	13					
11	14					
12	15					

SQL activity log - statements sent to the database

```
[18:23:24] [new connection] SELECT * FROM customer WHERE customer_id = 16;
[18:23:24] -> 0 rows: record does not exist in the database
[18:23:25] DELETE FROM customer WHERE customer_id = 1;
[18:23:25] -> REJECTED by MySQL: Cannot delete this record because other records still refer to it (FOREIGN KEY ... ON DELETE RESTRICT).
Cannot delete or update a parent row: a foreign key constraint fails ('hipcms`.`policy`, CONSTRAINT `fk_policy_customer` FOREIGN KEY
(`customer_id`) REFERENCES `customer` (`customer_id`) ON DELETE RESTRICT)
[18:23:30] SELECT * FROM dependant ORDER BY dependant_id;
[18:23:30] -> 12 row(s) returned
```

Figure 11.5: Dependant form and foreign-key deletion rejection in the SQL log.

The screenshot shows the HIPCMS interface with the 'Customers' panel. The panel displays a table of customers with columns for customer_id, first_name, last_name, phone, email, city, and state. There are 15 rows of customer data. To the right of the table are buttons for '+ Add Customer', '+ Add Dependant', 'Delete Customer', 'Delete Dependant', and 'Refresh'. Below the table is a section titled 'Dependants of selected customer' which is currently empty.

Customers

customer_id	first_name	last_name	phone	email	city	state
1	Arun	Sharma	9876500001	arun.sharma@example.com	Chennai	Tamil Nadu
2	Divya	Iyer	9876500002	divya.iyer@example.com	Chennai	Tamil Nadu
3	Karthik	Reddy	9876500003	karthik.reddy@example.com	Hyderabad	Telangana
4	Sneha	Patil	9876500004	sneha.patil@example.com	Pune	Maharashtra
5	Mohammed	Ali	9876500005	mohammed.ali@example.com	Hyderabad	Telangana
6	Lakshmi	Narayanan	9876500006	lakshmi.n@example.com	Chennai	Tamil Nadu
7	Rajesh	Kumar	9876500007	rajesh.kumar@example.com	Bengaluru	Karnataka
8	Priya	Venkatesh	9876500008	priya.venkatesh@example.com	Bengaluru	Karnataka
9	Anil	Kapoor	9876500009	anil.kapoor@example.com	Mumbai	Maharashtra
10	Sunita	Rao	9876500010	sunita.rao@example.com	Mumbai	Maharashtra
11	Vikram	Singh	9876500011	vikram.singh@example.com	New Delhi	Delhi
12	Neha	Gupta	9876500012	neha.gupta@example.com	New Delhi	Delhi
13	Suresh	Babu	9876500013	suresh.babu@example.com	Coimbatore	Tamil Nadu
14	Kavya	Krishnan	9876500014	kavya.krishnan@example.com	Kochi	Kerala
15	Ramesh	Iyer	9876500015	ramesh.iyer@example.com	Madurai	Tamil Nadu

Dependants of selected customer

Figure 11.6: Customers and dependants panel supplied as interface evidence.

11. UI Design and Screenshots — continued

HIPCMS							System Administrator ADMIN
Database Records	Customers & Dependants	Policies & Premiums	Claims	Appeals	Reports		
Report: Active Policies Run Report							
policy_number	policyholder	plan_name	sum_insured	start_date	end_date	status	
POL-2025-0009	Anil Kapoor	CorpShield Group Cover	2000000.00	2025-01-15	2025-12-31	ACTIVE	
POL-2025-0001	Arun Sharma	HealthSecure Family Floater Gold	1000000.00	2025-01-01	2025-12-31	ACTIVE	
POL-2025-0005	Mohammed Ali	HealthSecure Family Floater Gold	1000000.00	2025-01-10	2025-12-31	ACTIVE	
POL-2025-0007	Rajesh Kumar	HealthSecure Family Floater Gold	1000000.00	2025-05-01	2026-04-30	ACTIVE	
POL-2025-0002	Divya Iyer	HealthSecure Individual Basic	300000.00	2025-02-15	2026-02-14	ACTIVE	
POL-2025-0004	Sneha Patil	HealthSecure Individual Basic	300000.00	2025-03-01	2026-02-28	ACTIVE	
POL-2025-0008	Priya Venkatesh	HealthSecure Individual Basic	300000.00	2025-02-01	2026-01-31	ACTIVE	
POL-2025-0012	Neha Gupta	HealthSecure Individual Basic	300000.00	2025-07-01	2026-06-30	ACTIVE	
POL-2025-0006	Lakshmi Narayanan	SeniorCare Plus	500000.00	2025-04-01	2026-03-31	ACTIVE	
POL-2025-0013	Suresh Babu	SeniorCare Plus	500000.00	2025-03-15	2026-03-14	ACTIVE	
POL-2025-0010	Sunita Rao	SmartCare Micro	150000.00	2025-06-01	2026-05-31	ACTIVE	
POL-2025-0014	Kavya Krishnan	SmartCare Micro	150000.00	2025-08-01	2026-07-31	ACTIVE	

Report 'Active Policies' - 12 row(s).

DB: MySQL root@127.0.0.1:3306 / hipcms

Figure 11.7: Active Policies report with 12 records having stored ACTIVE status.

12. Implementation Details

12.1 Technology and architecture

Layer	Implementation
Presentation	Python Tkinter / ttk desktop widgets
Application	Form generation, required-field/date checks, ADMIN login, table allowlist
Connectivity	MySQL Connector/Python over 127.0.0.1:3306
Storage	hipcms database, fifteen relational tables
Integrity	Primary/foreign keys, UNIQUE, NOT NULL, CHECK and INSERT triggers

The user supplies database connection credentials and a separate application username/password. The application queries users for an active account and requires the ADMIN role. Password verification follows the supplied SHA-256 sample-data format; stronger password hashing is a future improvement.

On successful connection, the app reads table metadata using SHOW COLUMNS, generates forms for the chosen table, and omits generated IDs and created_at. Allowed CHECK values are displayed as choices where recognized. In the delivered Mac code, foreign-key values are entered as IDs; the reference dependant screenshot shows a richer parent-record dropdown.

SELECT statements refresh records from MySQL. INSERT and DELETE statements are parameterized, and the connection uses autocommit. Deletion requires confirmation. The server enforces constraints and triggers, while the GUI displays connector errors. SQL log values are omitted in the Mac package to avoid printing sensitive parameters.

12.2 Startup on macOS

```
cd ~/Downloads/HIPCMS_Original_Mac
source .venv/bin/activate
python app.py
```

Use host 127.0.0.1, port 3306, database hipcms and the project database account. The application sample login is admin / admin@123. Database-account passwords are intentionally excluded from this report. Existing data does not need to be imported each time the app starts.

12.3 Key Code Snippets

Database connection (from the delivered app)

```
db = mysql.connector.connect(
    host=v['Host'], port=int(v['Port']),
    user=v['MySQL user'], password=v['MySQL password'],
    database=v['Database'], connection_timeout=5,
    autocommit=True
)
```

Parameterized insertion

```
sql = (
    f'INSERT INTO `{table}` (' + ', '.join(cols)
    + ') VALUES (' + ', '.join(['%s' * len(vals)] + ')')
)
self.query(sql, vals, True)
```

Column names come from database metadata and the table is selected from the application allowlist. Form values are passed separately as parameters rather than concatenated into SQL.

Parameterized deletion

```
changed = self.query(
    f'DELETE FROM `{table}` WHERE `{pk}`=%s',
    (value,), True
)
```

Executing SQL and closing the cursor

```
c = self.db.cursor(dictionary=True)
try:
    c.execute(sql, params)
    return c.rowcount if write else c.fetchall()
finally:
    c.close()
```

Database-error feedback

```
except mysql.connector.Error as e:
    messagebox.showerror('Insert failed', str(e), parent=win)
```

13. Testing – Observed Results

The following results are supported by the submitted UI logs and Workbench screenshots. “Observed” refers to that evidence; it does not mean the report author independently ran the application on the student’s Mac.

ID	Test	Expected result	Evidence / actual result
T1	View customers	15 original customers	Observed: customer grid shows 15 rows.
T2	Insert Ananya Rao	One record added	Observed: customer_id 16; count 15 → 16; commit logged.
T3	Duplicate customer email	Reject duplicate email	Observed: log rejects ananya.rao@example.com under customer.email UNIQUE constraint.
T4	Delete demonstration customer	One record removed	Observed: log shows customer 16 deleted; count 16 → 15.
T5	Verify deleted ID	Zero matching records	Observed: new connection SELECT for customer 16 returns zero rows.
T6	Delete customer referenced by policy	Reject parent deletion	Observed: dependant screenshot log cites fk_policy_customer and ON DELETE RESTRICT.
T7	Presentation-II KIMS count	2 claims	Observed: Workbench result shows KIMS Hospital = 2.
T8	Check installed database	hipcms reachable; 15 customers	Observed: Workbench shows root@localhost, port 3306 and customer count 15.

The successful customer count does not by itself verify every row in every table. Sample-script totals and query outputs elsewhere in this report are labelled as derived results.

13.1 Additional Test Cases and Validation Gaps

The following cases are recommended tests based on the SQL and application code. Their outcomes are expected, not recorded live results.

ID	Input / action	Expected behaviour	Status
T9	Customer DOB today or later	INSERT trigger rejects birth date	Not evidenced
T10	Gender outside M/F/O	CHECK rejects value	Not evidenced
T11	Non-existent foreign-key ID	MySQL rejects child record	Not evidenced
T12	Claim for inactive policy	Coverage trigger rejects INSERT	Not evidenced
T13	Claim date outside policy dates	Coverage trigger rejects INSERT	Not evidenced
T14	Same policy/hospital/date duplicate	Duplicate-claim trigger rejects non-rejected duplicate	Not evidenced
T15	Treatment outside policy dates	Treatment trigger rejects INSERT	Not evidenced
T16	Settlement greater than sum insured	Settlement trigger rejects INSERT	Not evidenced
T17	Blank required UI field	Form prevents submission	Not evidenced
T18	Malformed date	UI requires YYYY-MM-DD	Not evidenced

Known gaps to address

A dependant's ownership is not checked against the policyholder by the supplied claim triggers. Partial payment handling, settlement approval prerequisites, cumulative coverage usage and status reconciliation after deletion require additional rules. The test plan should cover these behaviours before extending the project beyond a coursework demonstration.

14. Conclusion and Future Enhancements

14.1 Conclusion

HIPCMS provides a connected relational model for policy and claims management. Fifteen tables organize the core entities and events, while keys, constraints and triggers enforce selected integrity rules. The supplied dataset and SQL queries demonstrate customer-policy relationships, hospital claim counts, premium balances and settlements. The desktop UI supports record viewing, insertion and deletion with database feedback.

The supplied screenshots demonstrate the customer insertion/deletion sequence, duplicate-email rejection, foreign-key protection and the Presentation-II KIMS query. The report also distinguishes the initial conceptual design from the implemented SQL and separates observed test evidence from expected outcomes.

14.2 Future enhancements

- Add editing, search, pagination and complete role-specific interfaces.
- Reconcile claim and premium status after updates and deletions; calculate paid status from actual receipts.
- Require a valid current approval before settlement and enforce cumulative coverage limits.
- Validate that a claimed dependant belongs to the policyholder and is covered by the policy.
- Replace plain SHA-256 password verification with a modern salted password-hashing scheme.
- Add secure document uploads, audit trails and tested backup/restore procedures.
- Introduce date-aware policy expiry reports, renewal workflows and payment-gateway integration.

15. References

- [1] **Project Review I.** Presentation_1 (2).pdf, supplied by the student; problem identification, objectives and scope.
- [2] **Project Review II.** DBMS review 2 PDF.pdf, supplied by the student; conceptual ER diagram and initial relational schema.
- [3] **Final SQL sources.** Pasted text(2).txt and Pasted text (2).txt, supplied schema and expanded sample dataset.
- [4] **Project evidence.** Supplied HIPCMS screenshots and Workbench output for the Presentation-II KIMS query.
- [5] **Oracle, MySQL Reference Manual.** <https://dev.mysql.com/doc/refman/8.4/en/>
- [6] **Oracle, MySQL Connector/Python Developer Guide.** <https://dev.mysql.com/doc/connector-python/en/>
- [7] **Python Software Foundation, Tkinter documentation.** <https://docs.python.org/3/library/tkinter.html>
- [8] **Python Software Foundation, Using Python on macOS.** <https://docs.python.org/3/using/mac.html>

16. Appendix: GitHub Repository Link

Project repository supplied by the student:

<https://github.com/tanishqaboinipelly/DBMS-COURSE-PROJECT>

Suggested submission contents

- Final SQL schema and sample-data scripts.
- Presentation-I, Presentation-II and Presentation-III files.
- ER diagram, relational schema and this project report.
- Python application, requirements.txt and Mac startup instructions.
- UI screenshots, Presentation-II query output and testing evidence.

The link is included as provided. This report does not certify which files have already been uploaded to the repository. Keep private database passwords and local virtual-environment folders out of the submitted source files.

Project identification

Item	Value
Student	Tanishqa B
Roll number	25WU0102285
Project	Health Insurance Policy & Claims Management System
Project number	53
Course	Database Management System (DBMS)
Faculty	Dr. Kiranmayee
Academic year	2026-27
University	Woxsen University